

POLITECNICO DI TORINO

Cybersecurity National Lab – CINI

Master Degree Thesis

Master Degree in Cybersecurity

Design and Implementation of an Interactive Web-Based Cyber Range for Hardware Security Training



Supervisor

Alessandro Savino

Co-supervisor

Samuele Yves Cerini

Candidate

Davide Leone

Academic Year 2025/2026

Contents

1	Introduction	3
2	Background	6
2.1	Foundations of Hardware Security	6
2.1.1	The Role of Hardware in Cybersecurity	6
2.1.2	Hardware Security Threats and Adversary Motivations	6
2.1.3	Hardware Analysis Methodology	6
2.2	Embedded Systems Fundamentals	6
2.2.1	Architecture of Consumer IoT Devices	6
2.2.2	Common SoC Families and Components	6
2.2.3	Non-Volatile Storage and Flash Layout	6
2.2.4	MTD Subsystem	6
2.2.5	CPU Architectures	6
2.2.6	U-Boot and the Boot Chain	6
2.2.7	RTOS vs. Linux-Based Systems	6
2.3	Debug and Communication Interfaces	6
2.3.1	UART	6
2.3.2	SPI	6
2.4	Hardware and Software Tools	6
2.4.1	Hardware Instrumentation	6
2.4.2	Software Toolchain	6
2.4.3	Exploitability Factors in Consumer IoT Devices	6
3	State of the Art	7
3.1	Institutional Context	7
3.2	Hardware Security Training Landscape	7
3.2.1	Professional Conferences and Training	7
3.2.2	CTF Platforms and Hardware Competitions	7
3.2.3	IoT-Focused Training Resources	7
3.3	Remote Laboratories and Cyber Ranges	7
3.3.1	Remote Laboratories	7
3.3.2	Cyber Ranges	7
3.3.3	Circuit Simulators	7
3.3.4	Firmware Emulation Frameworks	7
3.4	Gap Analysis	7
3.5	Positioning of This Work	7

4	Contributions and Requirements Analysis	8
4.1	Motivations	8
4.2	Operational Context and Actors	8
4.3	Functional Requirements	8
4.3.1	PCB Simulation	8
4.3.2	Measurement and Connection Tools	8
4.3.3	Simulated Terminal	8
4.3.4	Exercise Structure	8
4.3.5	Authoring and Configuration	8
4.4	Non-Functional Requirements	8
	Accessibility and deployment.	8
	User experience.	8
	Architecture and security.	8
4.5	Use Cases	8
	Student use cases.	8
	Author use cases.	8
5	System Architecture	9
5.1	Technology Stack	9
5.2	General System Architecture	9
5.3	Data Model	9
5.3.1	Exercise Configuration	9
5.3.2	Coordinate System	9
5.3.3	Terminal Configuration	9
5.4	State Management	9
5.4.1	Dirty Tracking	9
5.4.2	Persistence Strategy	9
5.5	Terminal Simulation Design	9
5.5.1	Design Principles	9
5.5.2	Command Execution Pipeline	9
5.5.3	Filesystem Abstraction	9
5.5.4	Boot Sequence System	9
5.5.5	Progressive Flag System	9
5.6	Data Flow Between Authoring and Simulation	9
6	Implementation	10
6.1	Interactive PCB Viewer	11
6.1.1	Coordinate Conversion	11
6.1.2	Snap-to-Pin Mechanism	11
6.1.3	Wire Rendering	11
6.1.4	Magnifying Lens	11
6.2	Simulated Hardware Analysis Tools	11
6.2.1	Digital Multimeter	11
6.2.2	UART Adapter	11
6.2.3	SPI Firmware Dumper	11
6.3	Terminal Engine	11
6.3.1	Configuration Loader	11
6.3.2	Command Executor	11
6.3.3	Boot Sequence Animation	11
6.3.4	Tab Completion	11
6.3.5	State Persistence Across Mounts	11



6.4	Exercise Completion System	11
6.5	Authoring Panel	11
6.5.1	Visual Canvas Editor	11
6.5.2	Terminal Configuration Editor	11
6.5.3	Preset Management	11
6.6	API Layer	11
6.7	Security by Design	11
6.7.1	Architectural Mitigation: Client-Side Execution	11
6.7.2	Input Validation and Sanitisation	11
6.7.3	Path Traversal Prevention	11
6.7.4	Access Control	11
6.7.5	Secure Deployment	11
6.7.6	Cross-Site Scripting Prevention	11
6.7.7	Static Analysis and Type Safety	11
6.7.8	Summary of OWASP Top 10 Coverage	11
7	Validation: Case Study	12
7.1	Device Selection	12
7.2	Exercise Structure	12
7.2.1	Step 1: Hardware Identification	12
7.2.2	Step 2: UART Connection and Vulnerability Analysis	12
7.2.3	Step 3: SPI Firmware Dump	12
7.2.4	Terminal Phase: Embedded System Exploitation	12
7.3	Terminal Configuration	12
7.3.1	Terminal Challenge Walkthrough	12
7.4	Preliminary Observations	12
7.5	Authoring Workflow	12
8	Conclusions	13
8.1	Summary of Contributions	13
8.2	Requirements Assessment	13
8.3	Discussion	13
8.4	Limitations	13
8.5	Future Work	13

Chapter 1

Introduction



Consider a cybersecurity student who has just completed a network penetration testing course and wants to explore hardware security. **She** knows that real-world attacks against IoT devices—extracting firmware through debug interfaces, identifying hardcoded credentials on flash storage, exploiting bootloader misconfigurations—require hands-on interaction with physical circuit boards. Yet the path forward is daunting. A basic workstation with a multimeter, logic analyser, UART adapter, and SPI programmer **costs several hundred euros**. Expendable target devices must be sourced and periodically replaced. Professional training courses at conferences like Black Hat or through SANS carry fees between 3,000 and 8,000 EUR per participant [?]. **Unless her university maintains a dedicated** hardware security laboratory—and most do not—she has no affordable way to develop these skills.



This scenario is not hypothetical—it captures a structural problem in cybersecurity education with measurable consequences. The Mirai botnet of 2016 [?] exploited default credentials on hundreds of thousands of consumer IoT devices, causing some of the largest distributed denial-of-service attacks in internet history. Costin et al. [?] analysed over 32,000 firmware images from embedded devices and found that a majority contained at least one serious vulnerability—hardcoded passwords, unencrypted storage, or exposed debug interfaces. These are not exotic attack surfaces requiring nation-state resources; they are entry-level findings that a trained analyst can identify with a multimeter, a serial adapter, and basic command-line skills. The bottleneck is not the difficulty of the analysis but the scarcity of training environments in which students can develop these skills without access to a physical laboratory. On the software side, a mature ecosystem of Capture The Flag platforms has emerged: CTFd [?], PicoCTF [?], Hack The Box [?], and TryHackMe [?] deliver scalable, browser-accessible challenges covering web exploitation, binary analysis, cryptography, and reverse engineering. Hamari et al. showed that gamification elements—points, leaderboards, progressive unlocks—significantly increase engagement in learning contexts [?], and these platforms exploit that insight to great effect. Svabensky et al.’s comprehensive survey of hands-on cybersecurity training confirms that practical, interactive exercises produce measurably better learning outcomes than lecture-only instruction [?]. Yet their survey also reveals a stark asymmetry: virtually all existing platforms target software vulnerabilities. Hardware security training remains locked behind equipment costs, logistics, and physical access requirements.

Intermediate solutions exist but leave significant gaps. Remote laboratories such as iLab [?] and WebLab-Deusto [?] grant network access to real instruments, yet they demand dedicated server infrastructure and support only limited concurrency. Hardware CTF competitions like RHme [?] and the Embedded Security Challenge at CSAW [?] produce excellent pedagogical experiences, but they are episodic events that require physical

kits shipped to registered participants. Platforms like Microcorruption [?] and OWASP IoTGoat [?] focus narrowly—the former on a single microcontroller architecture, the latter on network-layer services—and neither simulates the physical board-level workflow that defines hardware analysis in practice: visually inspecting a PCB, probing test points with a multimeter, wiring a serial adapter with the correct crossover convention, dumping firmware through an SPI programmer. No existing solution combines these capabilities with a no-code authoring workflow in a zero-cost, browser-based package.

This thesis addresses that gap. We present **PCB-CTF**, a web-based interactive cyber range for hardware security training in CTF format. The platform enables a student to interact with a high-resolution photograph of a real printed circuit board, identify components through visual inspection and magnification, perform simulated electrical measurements with a digital multimeter, establish serial connections through a virtual UART adapter, extract firmware via a simulated SPI programmer, and explore a fully emulated embedded terminal—all within the browser, with no server-side code execution.

The first contribution is an architecture for hardware instrument simulation that runs entirely on the client side. The simulated multimeter, UART adapter, and SPI programmer produce realistic electrical feedback—voltage levels, impedance values, protocol-level validation of crossover wiring and role-based probe matching—while a normalised coordinate system ensures that interactive overlays scale responsively across display sizes.

The second contribution is a configurable terminal emulation system. We design a declarative configuration language that allows exercise authors to specify virtual filesystems, multi-stage boot sequences (including U-Boot and Linux login stages), custom commands with multiple output types, constraint-based access control, and a progressive flag discovery mechanism. The terminal reproduces the experience of interacting with a real embedded device through a synchronous command pipeline, without running any actual shell.

The third contribution is a no-code authoring interface. A visual exercise editor allows instructors to configure complete training scenarios—PCB image markup, component and pin placement, step and objective definition, terminal environment setup, and preset management—through a graphical interface. This lowers the barrier for content creation and enables educators without web development experience to produce and share exercises.

The fourth contribution is a validation through a real-world case study. We construct a complete exercise based on the hardware analysis of a TP-Link WR841N router, building directly on the vulnerability assessment methodology and device teardown conducted by Genova [?]. The exercise demonstrates that a multi-step investigation—from physical reconnaissance through UART connection to vulnerability discovery—can be faithfully reproduced in a fully virtual environment.

This work has been conducted within the ARTIC project, part of Spoke 4 (“Operating Systems and Virtualization Security”) of the SERICS (Security and Rights in Cyberspace) national research programme, funded under the Italian PNRR initiative.

Chapter 2 establishes the technical foundations of hardware security, embedded systems, and common debug interfaces. Chapter 3 surveys the current landscape of hardware security training, CTF platforms, and remote laboratories. Chapter 4 formalises the requirements that drive the platform design, while Chapter 5 details the system architecture and Chapter 6 covers implementation. Chapter 7 presents the TP-Link WR841N case study, and Chapter 8 draws conclusions and identifies directions for future work. Taken together, these chapters argue that accessible, high-fidelity hardware security training can be delivered through a browser-based platform at zero marginal cost per

student—and that doing so is not merely convenient but pedagogically necessary.

Chapter 2

Background

2.1 Foundations of Hardware Security

2.1.1 The Role of Hardware in Cybersecurity

2.1.2 Hardware Security Threats and Adversary Motivations

2.1.3 Hardware Analysis Methodology

2.2 Embedded Systems Fundamentals

2.2.1 Architecture of Consumer IoT Devices

2.2.2 Common SoC Families and Components

2.2.3 Non-Volatile Storage and Flash Layout

2.2.4 MTD Subsystem

2.2.5 CPU Architectures

2.2.6 U-Boot and the Boot Chain

2.2.7 RTOS vs. Linux-Based Systems

2.3 Debug and Communication Interfaces

2.3.1 UART

2.3.2 SPI

2.4 Hardware and Software Tools

2.4.1 Hardware Instrumentation

2.4.2 Software Toolchain

2.4.3 Exploitability Factors in Consumer IoT Devices

Chapter 3

State of the Art

3.1 Institutional Context

3.2 Hardware Security Training Landscape

3.2.1 Professional Conferences and Training

3.2.2 CTF Platforms and Hardware Competitions

3.2.3 IoT-Focused Training Resources

3.3 Remote Laboratories and Cyber Ranges

3.3.1 Remote Laboratories

3.3.2 Cyber Ranges

3.3.3 Circuit Simulators

3.3.4 Firmware Emulation Frameworks

3.4 Gap Analysis

3.5 Positioning of This Work

Chapter 4

Contributions and Requirements Analysis

4.1 Motivations

4.2 Operational Context and Actors

4.3 Functional Requirements

4.3.1 PCB Simulation

4.3.2 Measurement and Connection Tools

4.3.3 Simulated Terminal

4.3.4 Exercise Structure

4.3.5 Authoring and Configuration

4.4 Non-Functional Requirements

Accessibility and deployment.

User experience.

Architecture and security.

4.5 Use Cases

Student use cases.

Author use cases.

Chapter 5

System Architecture

5.1 Technology Stack

5.2 General System Architecture

5.3 Data Model

5.3.1 Exercise Configuration

5.3.2 Coordinate System

5.3.3 Terminal Configuration

5.4 State Management

5.4.1 Dirty Tracking

5.4.2 Persistence Strategy

5.5 Terminal Simulation Design

5.5.1 Design Principles

5.5.2 Command Execution Pipeline

5.5.3 Filesystem Abstraction

5.5.4 Boot Sequence System

5.5.5 Progressive Flag System

5.6 Data Flow Between Authoring and Simulation

Chapter 6

Implementation

6.1 Interactive PCB Viewer

6.1.1 Coordinate Conversion

6.1.2 Snap-to-Pin Mechanism

6.1.3 Wire Rendering

6.1.4 Magnifying Lens

6.2 Simulated Hardware Analysis Tools

6.2.1 Digital Multimeter

6.2.2 UART Adapter

6.2.3 SPI Firmware Dumper

6.3 Terminal Engine

6.3.1 Configuration Loader

6.3.2 Command Executor

6.3.3 Boot Sequence Animation

6.3.4 Tab Completion

6.3.5 State Persistence Across Mounts

6.4 Exercise Completion System

6.5 Authoring Panel

6.5.1 Visual Canvas Editor

6.5.2 Terminal Configuration Editor

6.5.3 Preset Management

6.6 API Layer

6.7 Security by Design

6.7.1 Architectural Mitigation: Client-Side Execution

6.7.2 Input Validation and Sanitisation

6.7.3 Path Traversal Prevention

Chapter 7

Validation: Case Study

7.1 Device Selection

7.2 Exercise Structure

7.2.1 Step 1: Hardware Identification

7.2.2 Step 2: UART Connection and Vulnerability Analysis

7.2.3 Step 3: SPI Firmware Dump

7.2.4 Terminal Phase: Embedded System Exploitation

7.3 Terminal Configuration

7.3.1 Terminal Challenge Walkthrough

7.4 Preliminary Observations

7.5 Authoring Workflow

Chapter 8

Conclusions

8.1 Summary of Contributions

8.2 Requirements Assessment

8.3 Discussion

8.4 Limitations

8.5 Future Work